

Condition Register Carry-Forward

Sample task overview

Long-horizon Harbor coding environment · CPU ML engineering

This document covers what the task is, what the agent has to do, how it is graded, how frontier models performed and what ships in the folder. The task files and full rollout traces are included alongside it.

Task identification

Field	Overview
Task ID	condition-register-carryforward
Category	CPU ML engineering: clinical risk scoring, point-in-time (as-of) feature construction and gated release operations
Scenario	A 30-day readmission scorer whose ward export repeats admissions and whose per-patient condition register does not match what was on file at each admission
Task format	Editable workspace, instruction, smoke tests, release-service CLI, protected release service, verifier, reference and alternate solutions
Real data	UCI Diabetes 130-US hospitals for years 1999–2008 (Strack et al., 2014; CC BY 4.0)
Compute	2 vCPU, 7 GB RAM, no GPU; network available to the agent (harness requirement); not used by the task; grading offline
Agent budget	9,000 seconds; verifier 2,400 seconds
Agent harness	mini-swe-agent, used for both models' rollouts
Deliverable	A driven release trajectory ending in cutover.commit, and the frozen artifact the environment replays over 14,248 encounters the agent never sees

Description

The agent inherits a clinical scoring pipeline with two open complaints. Every inpatient discharge is scored for readmission within 30 days and exported to the wards with a condition register next to each score. The ward administrator de-duplicates admissions in the export by hand every week, and clinical audit has found patients carrying the same conditions with scores far apart. The job log adds that a patient's condition list got shorter between two admissions. The backtest still reports about 0.65 AUC on a patient-grouped holdout and the smoke tests are green.

To solve it the agent has to do the **full engineer's job**: investigate the data and delivered artifacts, repair the register and the export, and drive the release through a gated service: rehearsal, deployment, certification, a forced remediation, canary and cutover. The release builder strips a required policy key from every release it ships, so the first certification always comes back dirty and must be diagnosed and remediated. What is

graded is the artifact frozen at cutover and the action log the root-owned service recorded, not a patch or a written claim.

The core technical difficulty is **defining the register correctly as of each admission**. The register owed for an encounter is the set of distinct diagnosis codes on file for that patient at or before that admission: accumulated per patient in encounter order across the archive and the batch, stopping at the admission, with codes exactly as charted. The delivered artifacts quietly contradict this, the familiar fix (group by patient and union everything) passes every visible check, and the gates check only the export's shape.

The incident the agent inherits

- **The export fans out.** A long-form register is merged onto the scores, so each admission appears once per code: the shipped pipeline writes 27,936 rows for 8,548 encounters. The job log's 25,562 counts one row per charted diagnosis, before the patient condition index is merged in.
- **A lifetime index, not a point-in-time one.** The register is read from a delivered condition index (112,493 rows, 32,577 patients) keyed on patient and code with no admission attached. Earlier admissions receive codes from later ones, and batch admissions miss codes from the same patient's earlier batch admissions.
- **A code table that folds codes.** The 867-row code table maps 34 charted codes onto a different register code (33 subdivisions of 250 onto 250, and 365.44 onto 365). The index manifest says codes are kept as charted, and the data card states that 250.83 and 250.8 are different codes.
- **A release builder that drops a policy key.** Every deploy.commit ships config.yaml without register.code_columns. The checkpoint reports policy_incomplete; incident.read names the family, and a wrong family costs an attempt and the diagnosis.
- **The archive is an input.** At grading the release is rerun against a shortened archive, so a memoised or vendored history fails.

What the agent has to do

The work combines a code repair with an ordered walk through the release service; reference trajectories take 17–24 accepted actions. Repair can come before or after a first deployment cycle: all four Claude Opus 5 runs fixed the code first and committed a single release.

Phase	What happens	Operations
1. Investigation	Read the policy, data card, manifest and job log; check the index and code table against the encounter columns; compare raw rows with what the working tree emits. Free and unlimited.	survey, ledger, pull_records, register_audit, status
2. Repair	As-of carry-forward over archive plus batch, codes as charted, archive read on every run; one row per encounter, register joined by the policy separator.	Edits to src/ (reference fix: register.py, serve.py)
3. Rehearsal and deployment	Run the backtest on the working tree, stage a code-only bundle and commit it as the release.	rehearse, deploy.prepare, deploy.commit
4. Forced recovery	First checkpoint is dirty (policy_incomplete). Read the diagnosis, apply that family, re-certify on the hidden batch.	checkpoint, incident.read, remediate --family config_drift, checkpoint
5. Canary and cutover	Open a canary (needs a clean post-remediation checkpoint), read the verdict, freeze.	canary.open, canary.read, cutover.commit

A checkpoint returns only clean or dirty plus a coarse signal. The budget is six checkpoints per episode (one refused before the release runs is free), three canaries and two remediation attempts per committed release; `deploy.commit` resets the attempts but reapplies the builder fault. Out-of-order operations are rejected.

Example prompt

Opening of the task instruction given to the agent:

We score every inpatient discharge for "will this patient be back inside 30 days", and the export goes to the wards with a condition register alongside each score so the care team can see what we think that patient is carrying. I inherited this pipeline from someone who moved on and I can't put my name on the next release. Two things came across with the handover and neither has an answer on it. The ward administrator says our export shows the same admission on several lines. ... And clinical audit pulled two patients last month who they say are carrying the same conditions, and our scores for them were nowhere near each other. ... What counts is the artifact you freeze at `cutover.commit` and how you got there - not a writeup and not a patch left sitting in the working tree. Don't tell me it's fixed; leave the trajectory that shows it.

The full instruction (`task/instruction.md`) describes the workspace, the output contract, the free observations, the gated sequence and the checkpoint, remediation and canary rules.

Time, QA and difficulty calibration

Difficulty was measured, not asserted: each model was run four times (`pass@4`) inside the real task environment with the `mini-swe-agent` harness, and every attempt was graded by the same verifier.

Model	Strict passes	Mean reward	Note
Claude Opus 5 (xhigh reasoning)	4/4	1.00	38–48 steps per run, 11–25 minutes of agent time
Grok 4.7 (xhigh reasoning)	2/4	0.50	41–75 steps per run, 5–15 minutes of agent time

Target profile: a clear frontier lead. The task is calibrated so that Grok 4.7 lands in a 25–50% band (1–2 of 4 attempts), solvable but discriminating, while the frontier Claude model solves it reliably. Here Claude Opus 5 solved 4 of 4 against Grok 4.7's 2 of 4, a 50-percentage-point lead in strict pass rate.

Pass bars. At grading the environment runs the frozen release over all 14,248 terminal encounters and compares each register with the verifier's own recomputation. Reward is 1 only if every clause holds:

Clause	Bar
Contract shape	<code>encounter_id</code> , score, conditions; one row per encounter; scores in [0, 1]
Coverage of terminal encounters	≥ 95%
Register agreement (exact set equality per encounter)	≥ 99%

Clause	Bar
Short-archive agreement (archive cut to its first 21,370 rows)	$\geq 99\%$
Discrimination (AUC against terminal labels)	≥ 0.56 (anti-degeneracy floor)
Operational path	Ten gated steps in order to cutover.commit, with dirty checkpoint $\rightarrow$ diagnosis $\rightarrow$ config_drift remediation $\rightarrow$ clean checkpoint $\rightarrow$ canary pass; ≥ 12 accepted actions, ≥ 2 observations

Per-attempt results (values that fail a bar in bold; — means the verifier stopped at the earlier clause):

Attempt	Result	Register agreement	Registers wrong	Short-archive agreement	AUC	Accepted actions
Opus 5 · 6LN5EPZ	Pass	100%	0	100%	0.6235	38
Opus 5 · 9jwPGRM	Pass	100%	0	100%	0.6235	20
Opus 5 · DQQWGVC	Pass	100%	0	100%	0.6235	18
Opus 5 · achMBut	Pass	100%	0	100%	0.6235	16
Grok 4.7 · 2MyrRjx	Fail	78.79%	3,022	—	—	36
Grok 4.7 · AHgRNZZ	Pass	100%	0	100%	0.6235	253
Grok 4.7 · KXmS7Rt	Fail	76.45%	3,355	—	—	32
Grok 4.7 · dGFGArS	Pass	100%	0	100%	0.6235	191

Steps and accepted actions measure different things. A step is one agent turn: a model call and the command it runs. An accepted action is one request the release service accepted, gated operations and free observations alike, as counted in the service's own log. A single step can issue many requests, so the two counts diverge: Grok 4.7's two passing runs scripted the service directly, one making 243 observation calls and the other committing 39 releases, which is why their accepted-action counts (253 and 191) exceed their step counts (68 and 75).

All eight attempts had 100% coverage and passed every operational check. Every passing release, across both models, produced the same AUC (0.6235) and score spread: once the register is right, the fixed policy model gives the same scores.

Why the model failed

Both failed Grok 4.7 attempts **completed the whole release correctly**. They fixed the export to one row per encounter, diagnosed and remediated the config_drift fault, took a clean checkpoint, passed the canary and cut over. Each failed on the register values alone, landing exactly on a negative control that isolates one half of the fix.

2MyrRjx kept the code table. It built a correct as-of carry-forward, stopping at each admission. Early on it listed the code table and saw the 34 remapped entries, but its final register still mapped charted codes

through that table and attached the folded index to batch admissions: 3,022 of 14,248 registers wrong (78.79%), the value pinned for the control with correct carry-forward and the folding table.

KXmS7Rt dropped the carry-forward. It saw that the lifetime index leaked later admissions' codes and removed the folding, then replaced the register with only the codes charted on the admission in hand, discarding the patient's earlier history: 3,355 registers wrong (76.45%), the value pinned for the control with codes as charted and no carry-forward.

That is by design. Checkpoints and canaries certify shape only, so a wrong register is certified clean. Getting the values right means reading the contract ("the conditions on file for the patient as of that admission"), the data card and the manifest closely, and checking each delivered artifact against the encounter columns. Grok 4.7's two passing attempts took 68 and 75 steps; Claude Opus 5 passed all four attempts in 38–48 steps.

Grading design

Reward is binary (1 only if the terminal, walk, recovery and depth checks all pass), but the verifier reports each component and the terminal metrics separately, so every attempt yields a dense diagnostic record rather than a single bit.

Component	Overview
Terminal value	The frozen release scores a later, unseen batch of 14,248 encounters. Each register is compared as a set with the verifier's recomputation from archive plus batch, so code order does not matter.
Archive re-read	The release is rerun with the archive cut to its first half; registers must follow the shorter history.
Discrimination floor	AUC ≥ 0.56 against held-out labels rules out degenerate scores only.
Operational path	The accepted-action log is replayed from root-owned state: ten gated steps in order, ending at <code>cutover.commit</code> , with a clean post-remediation checkpoint and a read, passing canary.
Forced recovery	The injected policy fault must be observed, diagnosed and repaired with the <code>config_drift</code> family.
Isolation	Releases carry only source and configuration (64 KB per file, 512 KB total). Candidate code runs unprivileged; hidden batches, labels and the probe are root-only. Candidate stdout and self-written reports are never read.

Failure, retries and limits

How we determine the agent has failed

Grading is deterministic and strictly binary: reward is 1.0 (pass) or 0.0 (fail). The verifier reads environment-owned state the agent cannot write: the release **frozen at `cutover.commit`**, replayed by the environment over 14,248 encounters the agent never sees, and the root-owned action log. The live workspace is never graded.

Reward is 1.0 only if every condition holds: the output contract and at least 95% coverage, register agreement of at least 99% on the full and the shortened archive, AUC of at least 0.56, the ten gated steps in order to `cutover.commit`, the forced policy fault diagnosed and repaired with the `config_drift` family, **and** the minimum depth of 12 accepted actions and 2 observations. Miss any one and the reward is 0.0.

Retries

- **No free retries.** Within an attempt the agent can fix, recommit and re-certify, but the feedback is lossy and rationed: six checkpoints per episode (one refused before the release runs is free), each returning only clean or dirty and a coarse signal, and three canaries.
- **Mistakes cost budget.** Each committed release allows two remediation attempts; a wrong family burns one along with the diagnosis. Recommitting resets the attempts but reapplies the builder fault.
- **One grading.** `cutover.commit` freezes a copy of the release; that frozen copy is graded once, after the attempt ends.
- **Across attempts:** `pass@4` means four fully independent attempts, each in a fresh container. They are independent samples used to measure the solve rate, not retries the agent controls. The target is Grok 4.7 in the 25–50% band (1–2 of 4) with the frontier Claude model well ahead; this task measured 4 of 4 for Claude Opus 5 and 2 of 4 for Grok 4.7.

Step and time limits

- **Time limit:** a hard 9,000 seconds (2 h 30 min) per attempt. An attempt still running at the limit is stopped.
- **Step limit:** none. The mini-swe-agent harness enforces no per-attempt step cap, so an attempt runs until the agent finishes or reaches the time limit.
- **Never cutting over is a fail, not an error.** An attempt that never reaches `cutover.commit`, or times out first, is graded 0, since there is no frozen release to grade. Genuine infrastructure failures, such as a container or harness fault, are reported separately as inconclusive and are never recorded as 0.

QA process

The task passes the same automated QA pipeline as the rest of the pack. Evidence for this task ships in the rollout folders:

- **Build and gates.** The Docker image built cleanly in both evaluation runs (digests `sha256:78f0efc7...` and `sha256:401fc18c...`, recorded in each `gold/gates.md`). In both runs the reference solution scores 1 and the untouched starting state scores 0.
- **Build-time guards.** The build fails if solution or verifier material or answer-like strings appear in the workspace, or if the service, probe and hidden directories are not root-only.
- **Three structurally different positive solutions.** A per-patient accumulator (oracle), a self-join on recorded-at $\leq$ described with codes written descending, and an expanding walk with no long form and first-seen order each score 1 with 100% agreement, via different routes through the gates. A fourth control with percentile-ranked scores and arbitrary code order also scores 1, so values are graded, not a copy of the reference.
- **Nineteen negative controls, all required to score 0 for a predicted reason:** untouched environment; full protocol with no fix; fixing only what the checkpoint names (66.99% agreement); carry-forward with the folding table (78.79%); codes as charted without carry-forward (76.45%); group-by-patient union (81.97%); correct register with meaningless scores; memoised archive (93.33% short-archive agreement); vendored archive; hunting for labels; forged stdout verdict; shortest-path fix; fault never diagnosed; guessed remediation family; and five malformed output files (empty, binary, truncated, oversized, misnamed). Pinned values are asserted within 5×10^{-4} .

Environment and execution

Python 3.12 CPU container (numpy 2.4.2, pandas 2.3.3, scikit-learn 1.7.2, scipy 1.17.0, PyYAML 6.0.3) with 10 GB of storage; network available to the agent (harness requirement); not used by the task; grading offline. The agent works unprivileged in /workspace/target/: the policy, encounter data, delivered index and code table, data card, six pipeline modules, scoring and backtest scripts, smoke tests, job log and env_cli.py. The release service runs as root behind a Unix socket, owns all release state and runs candidate code as a separate unprivileged user.

The runtime contract is a single command, scripts/score_batch.py --batch <csv> --out <csv>, writing one row per encounter with encounter_id, score and conditions.

Raw base data

Field	Overview
Source	UCI Machine Learning Repository, Diabetes 130-US hospitals for years 1999–2008 (dataset 296); Strack et al. (2014)
License	Creative Commons Attribution 4.0 International
Content	Real inpatient encounters for diabetic patients, 50 columns each; verbatim rows split by the corpus's own encounter ordering, none synthesised or edited
Visible data	42,741-encounter archive with the readmitted outcome; 8,548-encounter serving batch without it
Delivered artifacts	Condition index (112,493 rows, 32,577 patients) and code table (867 rows), both derived from the encounter columns
Held-out batches	Checkpoint 4,275, canary 1,424 and terminal 14,248 encounters, each later in encounter order; label readmitted = "<30" (about 11%)

Technical domains / relevant benchmarks

The task combines MLE-bench-style ML engineering (point-in-time feature construction, temporal leakage, clinical tabular data) with SWE-bench and DeepSWE-style repository repair and Terminal-Bench-style autonomous terminal execution. Its gated operations, rate-limited one-bit certification, hidden service state and injected release fault also exercise T-bench-style stateful tool use, applied to a production ML release rather than a simulated conversation.

Folder contents

Folder	Contents
task/	The complete Harbor task: instruction.md, task.toml, environment (Dockerfile, release service, hidden batches, grading probe, workspace), tests (verifier and probe shim), solution (reference and two alternate solutions) and _private (the negative-control battery with its expected outcomes).

Folder	Contents
Opus 5 pass@4/	agg/ holds the aggregate report and every trial's trajectory and verifier output for the four Claude Opus 5 rollouts; gold/ holds the build log, image digest and the oracle and no-op gate results.
Grok 4.7 pass@4/	The same layout for the four Grok 4.7 rollouts.

How the task was created

The real-data problem was chosen first: a public clinical corpus whose encounter numbers are the only chronology, which makes a point-in-time register both well defined and easy to get subtly wrong. The incident was built around it: the delivered artifacts are derived from the encounter columns so every claim they make can be checked, each fault is shown to be load-bearing by a control with a pinned agreement value, and the gates certify shape only. The task was then packaged with three structurally different solutions, the control battery, root-only verification and build-time guards.

Acknowledgement

I confirm that I have reviewed the tasks and believe they are realistic, technically substantive, clearly specified, and appropriately graded. They reflect genuine professional workflows and are not low-effort, artificial, or "slop" tasks.

Name: Suleiman Mahmood

Role: Co-Founder

Signature: /s/Suleiman Mahmood

Date: 9.28.26
